GITHUB · AGENTES · GOBERNANZA

Arquitectura de sistemas agénticos en GitHub

Autonomía dentro de límites explícitos, verificables y auditables



INTRODUCCIÓN

Los agentes cambian cómo se desarrolla el software

Un agente puede...

- Interpretar objetivos
- Analizar el estado de un repositorio
- Proponer soluciones
- Crear ramas y modificar archivos
- Crear Pull Requests
- Ejecutar procesos de validación
- Iterar a partir de pruebas, análisis de seguridad y revisiones de código



Más capacidades no convierten a un agente en un agente confiable

Debe diseñarse para que sus acciones sean:

01 **Limitadas**	02 **Observables**
03 **Auditables**	04 **Validables**
05 **Reversibles**	06 **Gobernadas por políticas**

LA IDEA CENTRAL

Proponer y ejecutar no es lo mismo que decidir

“Los agentes proponen y ejecutan dentro de límites definidos. Las personas y las políticas deciden qué se acepta.”



GitHub aporta los mecanismos para implementar esta arquitectura: trazabilidad, validación automática y control humano.

MECANISMOS DE GITHUB

Issues	Branches
Pull Requests	GitHub Actions
Rulesets y Branch Protection	CODEOWNERS
Environments	Secrets
Artifacts	Hooks

MCP — herramientas ampliadas para agentes

Plan y Planning: un artefacto y una capacidad

Plan

ARTEFACTO

Describe qué cambios se pretende realizar. Puede aparecer, por ejemplo, en la descripción de un Pull Request.

Normalmente incluye:

- Objetivo
- Alcance
- Archivos que se modificarán
- Pasos de implementación
- Riesgos
- Criterios de éxito
- Estrategia de rollback

Planning

CAPACIDAD DEL AGENTE

Generar o refinar el plan a partir de una tarea, un Issue o una instrucción.



Responsabilidades del agente a lo largo del SDLC

Etapa	Responsabilidad posible del agente	Artefacto principal
Planning	Definir alcance, pasos y criterios de éxito	Issue, descripción del PR
Implementation	Crear una rama y modificar código	Branch, commits, PR
Validation	Ejecutar pruebas y analizar resultados	Workflow runs, checks, artifacts
Deployment	Preparar el despliegue	Environments y approvals

Principio: cuanto mayor sea el riesgo de una operación, menor debería ser la autonomía del agente. Más autonomía en implementación y validación; controles más estrictos al llegar al despliegue.

RIESGO POR ÁREA

`docs/`	Bajo
`src/`	Medio
`.github/workflows/`	Alto
`infra/`	Alto
`Producción y secretos`	Crítico

Limitar qué puede modificar el agente

PERMITIDO

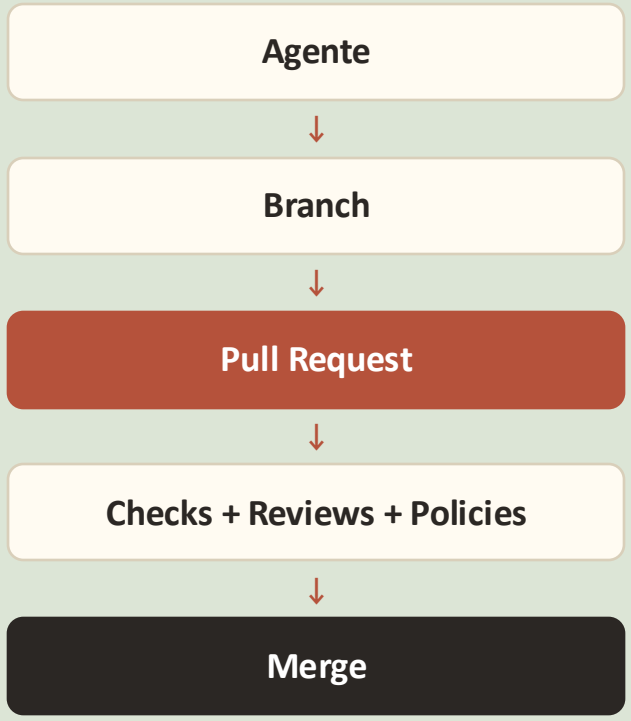
```
src/
docs/
package.json
package-lock.json
```

PROHIBIDO (SALVO AUTORIZACIÓN)

```
infra/
.github/workflows/
```

Evitar como regla general

- Push directo a main
- Modificación irrestricta de workflows
- Acceso innecesario a secretos
- Despliegues automáticos sobre producción
- Permisos de escritura que el agente no necesita



El Pull Request es un punto de control arquitectónico

Las tareas se definen como contratos de ejecución

Inputs

Qué información y límites recibe el agente

- Issue que describe una vulnerabilidad
- Archivos que puede modificar
- Directorios prohibidos
- Restricciones de seguridad
- Políticas del repositorio

Outputs

Qué debe producir

- Un plan
- Una rama
- Uno o varios commits
- Un Pull Request
- Evidencia de las pruebas realizadas

Success Criteria

Cómo sabemos que resolvió el problema

- Build, tests y lint exitosos
- Vulnerabilidad eliminada
- Ningún archivo fuera del alcance modificado
- Revisiones requeridas aprobadas

Que CI termine bien no significa que el objetivo se cumplió. Si el objetivo era eliminar una vulnerabilidad, el criterio debe comprobar que fue realmente eliminada, no solo que los tests pasan.

Separar Planning, Execution y Validation

01

Planning

¿Qué vamos a hacer y por qué?

REPRESENTACIÓN EN GITHUB

Issue, descripción del PR

02

Execution

¿Qué cambios concretos se realizaron?

REPRESENTACIÓN EN GITHUB

Branch + commits

03

Validation

¿Qué evidencia demuestra que los cambios son correctos?

REPRESENTACIÓN EN GITHUB

Checks + scans + artifacts + reviews

Así se puede revisar el objetivo antes de aceptar el impacto de los cambios.

Plan-first vs. Plan + Execution: cuándo se valida

Opción A · Plan-first

```
Tarea → Plan → Revisión humana → Aprobación →
Implementación → Validación
```

CUÁNDO UTILIZARLO

- Infraestructura, autenticación y seguridad
- Workflows y producción
- Componentes difíciles de revertir

Opción B · Plan + Execution

```
Tarea → Plan + código → Pull Request → Checks +
Reviews → Merge
```

CUÁNDO UTILIZARLO

- Riesgo bajo o medio
- Cambios fáciles de revertir
- Prioridad en velocidad e iteración
- Revisión simultánea de intención y código

Validación antes de generar cambios: reduce la exposición temprana.

Validación después de generar cambios, pero antes del merge: favorece la velocidad.

Una instrucción no es un control técnico

Instrucción al modelo

“No modifiques archivos todavía.”

Orienta el comportamiento del modelo, pero no impide técnicamente que la acción ocurra.

CONTROLES TÉCNICOS QUE SÍ LIMITAN

Tool allowlists	**Agentes de planificación read-only**	**Handoffs explícitos**
GitHub Rulesets	**Branch Protection**	**Required Checks**
CODEOWNERS	**Environment Approvals**	**Hooks**

Las instrucciones orientan el comportamiento. Los controles técnicos lo hacen cumplir.

El Pull Request como mecanismo de control

01 El agente crea una branch

02 El agente crea un Pull Request

03 El PR contiene el plan

04 GitHub Actions ejecuta validaciones

05 CODEOWNERS determina revisores

06 Required Reviews

07 Required Checks

08 **Merge permitido**

PLANTILLA: .github/pull_request_template.md

```
## Plan
- **Objetivo:**  **Alcance:**  **Archivos afectados:**  **Pasos:**
## Criterios de éxito
- [ ] Build exitoso   - [ ] Tests exitosos
- [ ] Análisis de seguridad revisado
- [ ] Cambios limitados al alcance definido
## Riesgos  /  ## Mitigaciones  /  ## Rollback
## Evidencia
- Workflow runs, resultados de tests y de seguridad
## Revisión
- [ ] Plan revisado  - [ ] Reviews y checks requeridos
```

El agente puede crear y modificar código sin autoridad para decidir unilateralmente si debe incorporarse. La plantilla hace la propuesta fácil de entender y revisar.

Actions convierte criterios de éxito en controles

```yaml
on:
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  security-analysis:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - name: Initialize analysis
        uses: github/codeql-action/init@v3
      - name: Analyze
        uses: github/codeql-action/analyze@v3
```



El éxito deja de ser algo asumido por el agente y pasa a ser comprobado por el sistema.

Revisores según el área y autonomía según el riesgo

CODEOWNERS

```
/security/          @security-team
/.github/workflows/ @platform-team
/infra/             @platform-team
*                   @core-team
```

Un cambio en infra/ puede requerir automáticamente al equipo responsable. Clave cuando los agentes pueden tocar partes sensibles del repositorio.

Riesgo	Ejemplo	Autonomía
Bajo	docs/, formato	Automerge después de validaciones
Medio	src/, dependencias	PR + checks + review
Alto	infra/, workflows	CODEOWNERS + múltiples reviews
Crítico	Producción, secretos	Approval humano obligatorio

La autonomía es una decisión arquitectónica, no una consecuencia accidental de los permisos disponibles.

Aprobación humana y secretos acotados

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - run: echo "Deploying to production..."
```

El agente puede

- Preparar el cambio
- Crear el PR
- Ejecutar tests
- Preparar el despliegue

Pero no necesariamente

Autorizar el despliegue a producción



Gestión de secretos

NUNCA EN

- Archivos de instrucciones
- Configuración versionada o código
- Workflow YAML en texto plano

DEBEN

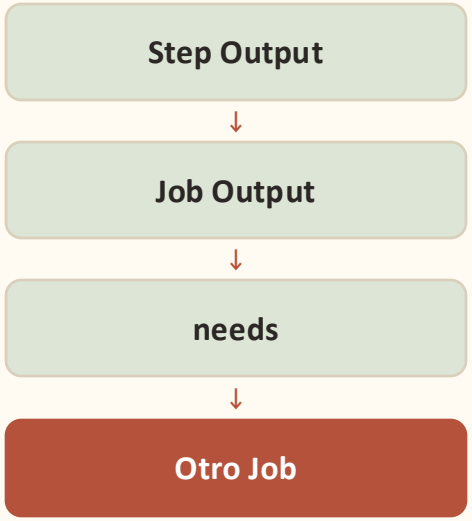
- Inyectarse en runtime
- Estar solo donde se necesitan, con alcance mínimo
- Usar límites protegidos, como environments

El entorno del agente puede tener su propio límite de secretos: no asumir que hereda los de CI.

Outputs, contextos y defensive gating

Outputs como contratos

Información tratada como datos estructurados



Step output: entre pasos del mismo job. Job output: entre jobs, con needs.

Contextos clave

`github.*`

Contexto del evento: ¿qué lo disparó? (github.event_name)

`vars.*`

Configuración centralizada y reutilizable

`env.*`

Configuración del runtime (job o step)

Defensive gating

```
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  validate-pr:
    if: github.event_name
        == 'pull_request'
```

Aunque el workflow pueda iniciarse a mano, ese job solo corre con el contexto de un Pull Request.

Evidencia visible de cada acción del agente

EL MÍNIMO DE EVIDENCIA DEBE RESPONDER

1. *¿Qué intentó hacer el agente?*
2. *¿Qué modificó?*
3. *¿Qué validaciones se realizaron?*
4. *¿Cuál fue el resultado?*
5. *¿Quién aprobó el cambio?*

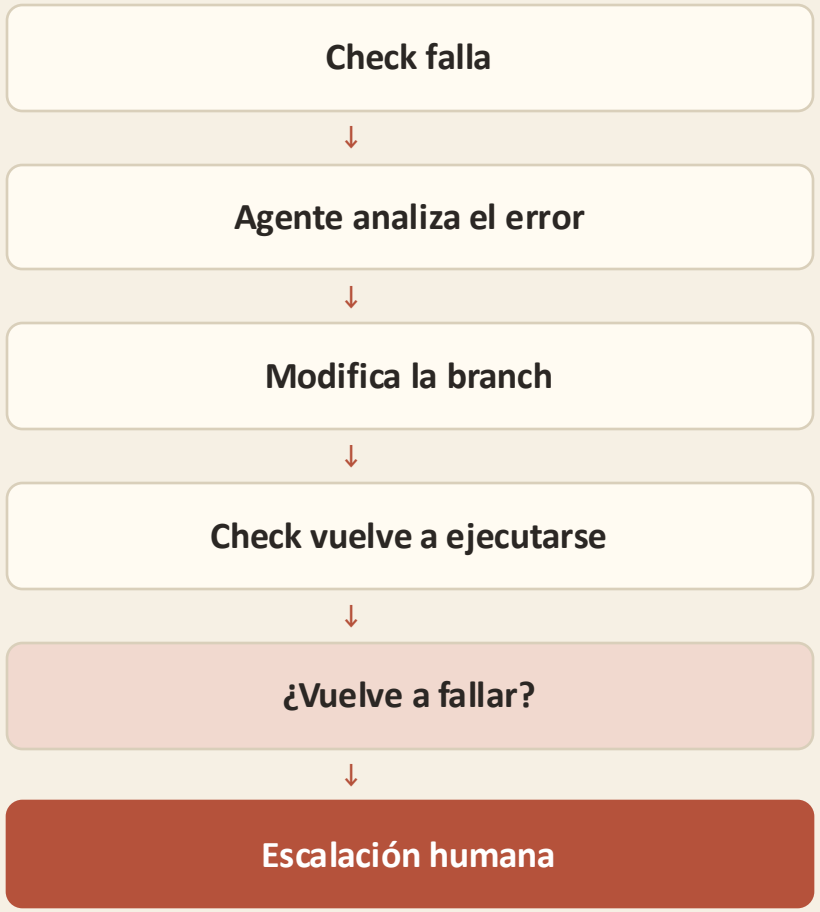
DÓNDE QUEDA LA EVIDENCIA

Pull Requests	Timeline del PR	Commits
Historial de branches	Workflow runs	Job logs
Required Checks	Security scans	Workflow artifacts
Reviews	Approvals	

```yaml
- name: Upload test results
  uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: results/
```

Relacionar la evidencia con el workflow run y el commit permite saber qué ejecución produjo el resultado y sobre qué estado del código.

Diseñar asumiendo que habrá fallos



La arquitectura contempla

- Retries y límites de reintentos
- Escalación humana
- Rollback
- Evidencia del fallo
- Least privilege

Una escalación muestra

- Qué falló
- Qué intentó el agente
- Qué evidencias existen
- Qué recomienda hacer después

Los fallos van a ocurrir. *Los reintentos acotados y la escalación humana evitan loops infinitos.*

Cada agente recibe solo las capacidades que necesita

Planning Agent

SOLO LECTURA

- Analizar e investigar
- Crear un plan
- No modifica el repositorio

Implementation Agent

CON ESCRITURA

- Crear branches
- Modificar archivos y crear commits
- Actualizar Pull Requests

Review Agent

SOLO LECTURA

- Analizar código y tests
- Revisar resultados y políticas
- Se mantiene read-only

MCP y el blast radius

Qué herramientas se habilitan	Qué operaciones permite cada una	Cómo se autentican	Qué secretos usan	Qué agente puede usarlas

Preferir listas explícitas de herramientas autorizadas. Ampliar MCP es un cambio sensible de gobernanza.

Hooks y least privilege como guardrails



INDEPENDIENTES DEL RAZONAMIENTO DEL MODELO

Pre-action hooks
Validan comandos, detectan patrones peligrosos y bloquean operaciones.

Post-action hooks
Registran operaciones, generan auditoría y capturan resultados.

Error hooks
Registran fallos, generan alertas e inician escalaciones.

```
permissions:
  contents: read
  pull-requests: write
```

Least privilege en Actions: leer el repositorio e interactuar con PRs, sin escritura innecesaria.

17 · ARQUITECTURA COMPLETA

Arquitectura de un sistema agéntico gobernado

1 · PLANIFICACIÓN

Tarea → Planning Agent → Plan → Human Approval → Implementation Agent

2 · EJECUCIÓN Y VALIDACIÓN

Branch → Pull Request → Tests · Security · CODEOWNERS → Required Checks → Human Review

3 · ENTREGA

Merge → Environment Gate → Deploy

El agente nunca decide solo el merge ni el despliegue

Control del agente

Herramientas permitidas, scope, MCP y least privilege

Control del repositorio

Branches, Pull Requests, CODEOWNERS, Rulesets y Branch Protection

Control de ejecución

Actions, Required Checks, Environments, Approvals, Hooks y Secrets

Diez principios fundamentales

01 **No trabajar directamente sobre main**
Branch → Pull Request → Validation → Merge

02 **El plan debe ser visible**
Qué, por qué, qué archivos y cómo se mide el éxito

03 **Planning y Execution diferenciables**
Especialmente en operaciones de alto riesgo

04 **Instrucciones ≠ controles técnicos**
“No modificar producción” no impide hacerlo

05 **Criterios de éxito verificables**
Tests PASS · Security PASS · Review APPROVED · Scope VALID

06 **La evidencia debe quedar registrada**
Task → Plan → Commit → Workflow Run → Evidence → Review

07 **La autonomía depende del riesgo**
Menor riesgo, más autonomía; mayor riesgo, más control humano

08 **Asumir que habrá fallos**
Retries limitados, escalación, rollback, logs y artifacts

09 **Aplicar Least Privilege**
Cada agente, workflow y herramienta, con permisos mínimos

10 **Agentes dentro de límites**
Observables, verificables y gobernados

CONCLUSIÓN

Diseñar agentes confiables es construir una arquitectura donde la autonomía está rodeada de controles.

GitHub aporta los componentes: Pull Requests, Actions, Required Checks, Rulesets, Branch Protection, CODEOWNERS, Environments, Secrets, Artifacts, Hooks y control de herramientas.

“Un sistema agéntico confiable permite autonomía dentro de límites explícitos, verificables y auditables.”

01 Objetivo

02 Plan

03 Ejecución controlada

04 Pull Request

05 Validación automática

06 Revisión humana

07 Políticas

08 Aprobación

09 Merge / Deployment